Name: Vansh Sharma

Roll NO: 2401010045

Section-B

Course:B.tech CseCore

Assignment: 1

❖ **Aim :**
To build a **dynamic To-Do List web application** using **HTML, CSS, JavaScript**, and a **database- backed backend** to **add, edit, delete, and manage tasks permanently** with full CRUD functionality.

❖ **Tasks :**

- **Project Setup & Introduction** – Create project folders (frontend & backend) and link HTML, CSS, JS files.
- **UI Layout (HTML)** – Input field for task, optional description, priority dropdown, Add Task button, task list section, filter buttons (All/Active/Completed), and counters.
- **Styling (CSS)** – Centered, mobile-friendly layout, completed task styling, priority badges, button hover effects.
- **Frontend Interactivity (JS)** – Add, edit, delete, complete tasks; filter tasks; show counts; input validation.
- **Backend + Database** – Use Node.js + Express + SQLite (or

other backend) to persist tasks.

- **REST API Implementation** – CRUD endpoints: GET, POST, PUT, PATCH, DELETE for tasks.
- **Bonus Features (Optional)** – Search, sort, due dates, localStorage fallback, JSON export, basic authentication.

❖ **Tech Stack Used :**

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Node.js, Express
- **Database:** SQLite (or MySQL/other as per setup)
- **APIs:** RESTful APIs for CRUD operations

❖ Html Code :

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<title>Advanced Task Manager</title>
<link rel="stylesheet" href="style.css">
<link
href="https://fonts.googleapis.com/css2?family=Plus+Jak
arta+Sans:wght@400;600;800&display=swap"
rel="stylesheet">
<script src="https://unpkg.com/lucide@latest"></script>
</head>
<body>
<div class="dashboard-container">
<aside class="sidebar">
<div class="brand">
<div class="logo-box"><i data-lucide="shield-
check"></i></div>
<span>TaskMaster Pro</span>
</div>

<div class="nav-group">
<p class="nav-label">Main Menu</p>
<button class="nav-btn active" onclick="setFilter('all',
this)"><i data-lucide="layout"></i> All Tasks</button>
<button class="nav-btn" onclick="setFilter('active',
this)"><i data-lucide="activity"></i> Pending</button>
```

```
<button class="nav-btn" onclick="setFilter('completed',
this)"><i data-lucide="check-square"></i>
Completed</button>
</div>

<div class="extra-actions">
<p class="nav-label">Data Management</p>
<button class="nav-btn" onclick="exportTasks()"><i
data-lucide="download"></i> Export JSON</button>
</div>

<div class="stats-card">
<div class="progress-circle-container">
<svg viewBox="0 0 36 36" class="circular-chart">
<path class="circle-bg" d="M18 2.0845 a 15.9155
15.9155 0 0 1 0 31.831 a 15.9155 15.9155 0 0 1 0 -31.831"
/>
<path id="circleFill" class="circle" stroke-dasharray="0,
100" d="M18 2.0845 a 15.9155 15.9155 0 0 1 0 31.831 a
15.9155 15.9155 0 0 1 0 -31.831" />
</svg>
<div class="percentage" id="percentNum">0%</div>
</div>
<p id="progressLabel">Productivity Score</p>
</div>
</aside>

<main class="main-content">
<header class="header">
<div class="search-bar">
<i data-lucide="search"></i>
```

```
<input type="text" id="searchInput" placeholder="Search
by keyword..." oninput="renderTasks()">
</div>
<div class="user-profile">
<div class="notif-bell"><i data-lucide="bell"
style="width:18px"></i></div>
<div class="avatar">V</div>
</div>
</header>

<section class="action-area">
<div class="entry-card">
<input type="text" id="taskTitle" placeholder="Task Title"
maxlength="50">
<textarea id="taskDesc" placeholder="Describe your
objective..."></textarea>
<div class="entry-actions">
<div class="meta-pickers">
<select id="priority">
<option value="Low">Low</option>
<option value="Medium" selected>Medium</option>
<option value="High">High</option>
</select>
</div>
<button class="add-task-btn"
onclick="handleAddTask()">Create Task</button>
</div>
</div>

<div class="list-container">
<div class="list-header">
```

```
<h3 id="currentView">Task Overview</h3>
<span id="taskCounter">0 Tasks</span>
</div>
<ul id="taskList"></ul>
</div>
</section>
</main>
</div>

<div id="editModal" class="modal-overlay">
<div class="modal-card">
<h3>Update Task</h3>
<input type="text" id="editTitle">
<textarea id="editDesc"></textarea>
<select id="editPriority">
<option value="Low">Low</option>
<option value="Medium">Medium</option>
<option value="High">High</option>
</select>
<div class="modal-footer">
<button class="secondary-btn"
onclick="closeEditModal()">Cancel</button>
<button class="primary-btn"
onclick="saveTaskEdit()">Save Changes</button>
</div>
</div>
</div>

<script src="script.js"></script>
</body>
</html>
```

- CSS Code :

```
:root {
--primary: #4f46e5;
--primary-soft: #eef2ff;
--bg-page: #f9fafb;
--bg-card: #ffffff;
--text-dark: #111827;
--text-gray: #6b7280;
--border: #e5e7eb;
}

* { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Plus Jakarta Sans', sans-serif; }
body { background: var(--bg-page); color: var(--text-dark); height: 100vh; overflow: hidden; }
.dashboard-container { display: flex; height: 100vh; }

/* Sidebar & Brand */
.sidebar { width: 280px; background: white; border-right: 1px solid var(--border); padding: 30px 20px; display: flex; flex-direction: column; }
.brand { display: flex; align-items: center; gap: 12px; font-weight: 800; font-size: 18px; color: var(--primary); margin-bottom: 40px; }
.logo-box { background: var(--primary); color: white; padding: 8px; border-radius: 10px; }

.nav-label { font-size: 11px; font-weight: 700; color: #9ca3af; text-transform: uppercase; letter-spacing: 1px; margin: 20px 0 10px; }
```

```
.nav-btn { width: 100%; border: none; background: transparent; display: flex; align-items: center; gap: 12px; padding: 12px; border-radius: 12px; cursor: pointer; color: var(--text-gray); font-weight: 600; transition: 0.2s; }
.nav-btn:hover { background: var(--bg-page); color: var(--primary); }
.nav-btn.active { background: var(--primary-soft); color: var(--primary); }

/* Circular Progress */
.stats-card { margin-top: auto; padding-top: 20px; text-align: center; border-top: 1px solid var(--border); }
.progress-circle-container { position: relative; width: 100px; margin: 0 auto 10px; }
.circular-chart { display: block; width: 100%; }
.circle-bg { fill: none; stroke: #eee; stroke-width: 3.8; }
.circle { fill: none; stroke: var(--primary); stroke-width: 3.8; stroke-linecap: round; transition: stroke-dasharray 0.5s ease; }
.percentage { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); font-weight: 800; font-size: 14px; }
#progressLabel { font-size: 12px; font-weight: 600; color: var(--text-gray); }

/* Main Content */
.main-content { flex: 1; display: flex; flex-direction: column; overflow-y: auto; }
.header { padding: 20px 40px; border-bottom: 1px solid var(--border); background: white; display: flex; justify-
```

```
content: space-between; align-items: center; }
.search-bar { position: relative; width: 350px; }
.search-bar i { position: absolute; left: 12px; top: 50%;
transform: translateY(-50%); color: #9ca3af; width: 18px;
}
.search-bar input { width: 100%; padding: 10px 40px;
border-radius: 12px; border: 1px solid var(--border);
outline: none; background: #f3f4f6; }

.avatar { width: 35px; height: 35px; background: var(--
primary); color: white; border-radius: 50%; display: flex;
align-items: center; justify-content: center; font-weight:
800; }

/* Input Area */
.action-area { padding: 40px; max-width: 900px; margin:
0 auto; width: 100%; }
.entry-card { background: white; border-radius: 20px;
padding: 25px; box-shadow: 0 10px 25px
rgba(0,0,0,0.03); border: 1px solid var(--border);
margin-bottom: 40px; }
#taskTitle { width: 100%; border: none; font-size: 22px;
font-weight: 700; outline: none; margin-bottom: 10px; }
#taskDesc { width: 100%; border: none; font-size: 14px;
outline: none; resize: none; height: 50px; color: var(--
text-gray); }
.entry-actions { display: flex; justify-content: space-
between; border-top: 1px solid var(--border); padding-
top: 15px; }
.add-task-btn { background: var(--primary); color: white;
border: none; padding: 12px 25px; border-radius: 12px;
```

```
font-weight: 700; cursor: pointer; transition: 0.3s; }
.add-task-btn:hover { background: #3730a3; transform:
scale(1.02); }

/* List */
.list-header { display: flex; justify-content: space-
between; margin-bottom: 20px; align-items: center; }
li { background: white; padding: 20px; border-radius:
16px; margin-bottom: 15px; border: 1px solid var(--
border); display: flex; justify-content: space-between;
align-items: flex-start; transition: 0.2s; }
li:hover { box-shadow: 0 5px 15px rgba(0,0,0,0.05);
border-color: var(--primary); }
li.done { opacity: 0.5; background: #f8fafc; }

.task-main { display: flex; gap: 15px; }
.check-btn { width: 24px; height: 24px; border: 2px solid
var(--border); border-radius: 8px; cursor: pointer;
transition: 0.2s; display: flex; align-items: center; justify-
content: center; }
.done .check-btn { background: var(--primary); border-
color: var(--primary); color: white; }

.task-info strong { display: block; font-size: 16px;
margin-bottom: 5px; }
.task-info p { font-size: 13px; color: var(--text-gray); }
.timestamp { font-size: 11px; color: #9ca3af; margin-
top: 8px; display: block; }

.badge { font-size: 10px; font-weight: 800; padding: 4px
10px; border-radius: 6px; text-transform: uppercase;
```

```
margin-top: 10px; display: inline-block; }
.High { background: #fef2f2; color: #dc2626; }
.Medium { background: #fffbeb; color: #d97706; }
.Low { background: #f0fdf4; color: #16a34a; }

.actions { display: flex; gap: 10px; }
.actions button { background: none; border: none;
cursor: pointer; color: var(--text-gray); padding: 5px; }
.actions button:hover { color: var(--primary); }

/* Modal */
.modal-overlay { position: fixed; inset: 0; background:
rgba(0,0,0,0.4); backdrop-filter: blur(5px); display: none;
align-items: center; justify-content: center; z-index:
1000; }
.modal-card { background: white; padding: 30px;
border-radius: 20px; width: 450px; }
.modal-card input, .modal-card textarea, .modal-card
select { width: 100%; padding: 12px; margin-bottom:
15px; border: 1px solid var(--border); border-radius:
10px; outline: none; }
.modal-footer { display: flex; justify-content: flex-end;
gap: 10px; }
```

- JS Code :

- Script.js :

```
const API_URL = 'http://localhost:3000/tasks';
let tasks = [];
let currentFilter = 'all';
let currentEditId = null;

window.onload = () => {
```

```
fetchTasks();
lucide.createIcons();
};

async function fetchTasks() {
const res = await fetch(API_URL);
tasks = await res.json();
renderTasks();
}

async function handleAddTask() {
const title =
document.getElementById('taskTitle').value.trim();
const description =
document.getElementById('taskDesc').value.trim();
const priority =
document.getElementById('priority').value;

if (!title) return alert("Task title required!");

await fetch(API_URL, {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ title, description, priority })
});

document.getElementById('taskTitle').value = '';
document.getElementById('taskDesc').value = '';
fetchTasks();
}
```

```
async function toggleStatus(id, current) {
await fetch(`${API_URL}/${id}/status`, {
method: 'PATCH',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ isDone: current === 0 ? 1 : 0 })
});
fetchTasks();
}

async function deleteTask(id) {
if (confirm("Permanently delete this?")) {
await fetch(`${API_URL}/${id}`, { method: 'DELETE' });
fetchTasks();
}
}

// EDIT LOGIC
function openEditModal(id, title, desc, priority) {
currentEditId = id;
document.getElementById('editTitle').value = title;
document.getElementById('editDesc').value = desc;
document.getElementById('editPriority').value = priority;
document.getElementById('editModal').style.display =
'flex';
}

function closeEditModal() {
document.getElementById('editModal').style.display =
'none'; }

async function saveTaskEdit() {
```

```
await fetch(`${API_URL}/${currentEditId}`, {
method: 'PUT',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({
title: document.getElementById('editTitle').value,
description: document.getElementById('editDesc').value,
priority: document.getElementById('editPriority').value
})
});
closeEditModal();
fetchTasks();
}

// EXPORT TO JSON (Bonus Feature)
function exportTasks() {
const dataStr = "data:text/json;charset=utf-8," +
encodeURIComponent(JSON.stringify(tasks));
const downloadAnchor = document.createElement('a');
downloadAnchor.setAttribute("href", dataStr);
downloadAnchor.setAttribute("download",
"tasks_backup.json");
downloadAnchor.click();
}

function renderTasks() {
const list = document.getElementById('taskList');
const query =
document.getElementById('searchInput').value.toLowerCa
se();
list.innerHTML = '';
```

```
const filtered = tasks.filter(t => {
const matchesSearch =
t.title.toLowerCase().includes(query);
if (currentFilter === 'active') return t.isDone === 0 &&
matchesSearch;
if (currentFilter === 'completed') return t.isDone === 1
&& matchesSearch;
return matchesSearch;
});

filtered.forEach(t => {
const date = new
Date(t.createdAt).toLocaleDateString('en-GB', { day:
'numeric', month: 'short' });
const li = document.createElement('li');
li.className = t.isDone ? 'done' : '';
li.innerHTML = `
<div class="task-main">
<div class="check-btn" onclick="toggleStatus(${t.id},
${t.isDone})">
${t.isDone ? '<i data-lucide="check"
style="width:14px"></i>' : ''}
</div>
<div class="task-info">
<strong>${t.title}</strong>
<p>${t.description || ''}</p>
<span class="timestamp"><i data-lucide="calendar"
style="width:10px; display:inline"></i> ${date}</span>
<span class="badge ${t.priority}">${t.priority}</span>
</div>
</div>
```

```
<div class="actions">
<button onclick="openEditModal(${t.id}, '${t.title}',
'${t.description}', '${t.priority}')"><i data-lucide="edit-
3"></i></button>
<button onclick="deleteTask(${t.id})"
style="color:#ef4444"><i data-lucide="trash-
2"></i></button>
</div>
`;
list.appendChild(li);
});

lucide.createIcons();
updateStats();
}

function updateStats() {
const total = tasks.length;
const completed = tasks.filter(t => t.isDone).length;
const percent = total > 0 ? Math.round((completed /
total) * 100) : 0;

// Update Circular Chart
document.getElementById('circleFill').setAttribute('stroke-
dasharray', `${percent}, 100`);
document.getElementById('percentNum').innerText =
`${percent}%`;
document.getElementById('taskCounter').innerText =
`${total} Total Tasks`;
}
```

```
function setFilter(f, btn) {
currentFilter = f;
document.querySelectorAll('.nav-btn').forEach(b =>
b.classList.remove('active'));
btn.classList.add('active');
document.getElementById('currentView').innerText = f
=== 'all' ? 'Task Overview' : f.charAt(0).toUpperCase() +
f.slice(1);
renderTasks();
}
```

❖ Package.json:

```
{

  "name": "dynamic-todo-backend",

  "version": "1.0.0",

  "description": "Persistent Todo List with SQLite",

  "main": "server.js",

  "scripts": {

    "start": "node server.js"

  },

  "dependencies": {

    "cors": "^2.8.5",

    "express": "^4.18.2",
```

```
    "sqlite3": "^5.1.6"

  }

}
```

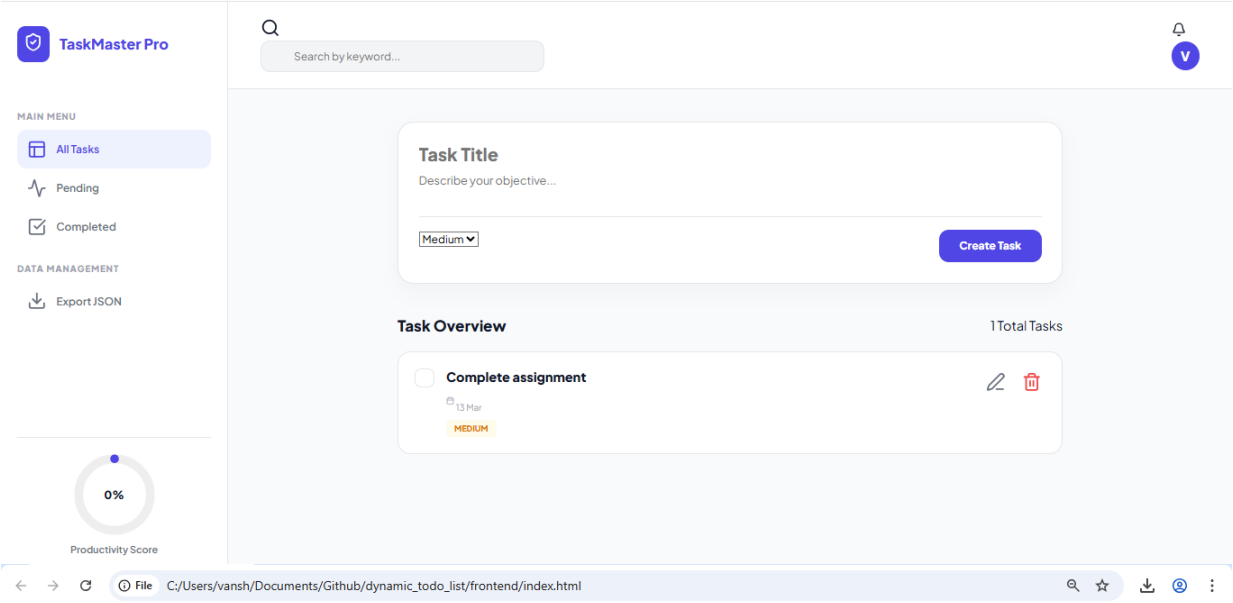
❖ Server.js:

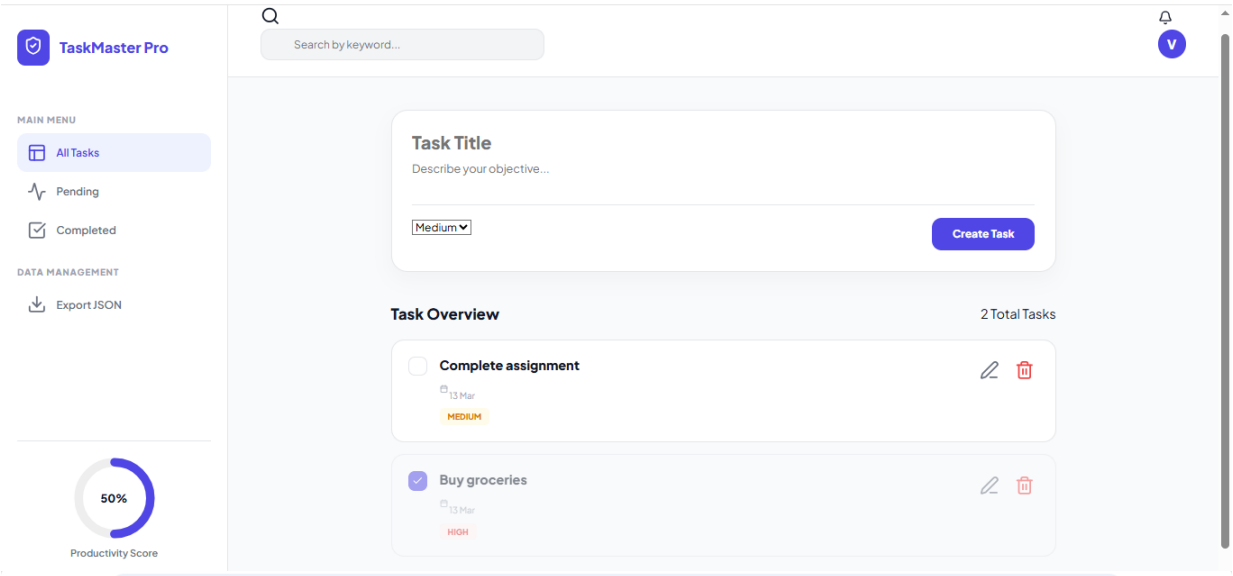
```
const express = require('express');
const sqlite3 = require('sqlite3').verbose();
const cors = require('cors');

const app = express();
app.use(express.json());
app.use(cors());

const db = new sqlite3.Database('./db.sqlite');

db.serialize(() => {
    db.run(`CREATE TABLE IF NOT EXISTS tasks (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        description TEXT,
        priority TEXT DEFAULT 'Medium',
        isDone INTEGER DEFAULT 0,
        createdAt DATETIME DEFAULT
CURRENT_TIMESTAMP
    )`);
});

app.get('/tasks', (req, res) => {
    db.all("SELECT * FROM tasks ORDER BY isDone ASC, id
DESC", [], (err, rows) => {
        if (err) return res.status(500).json({ error: err.message
```

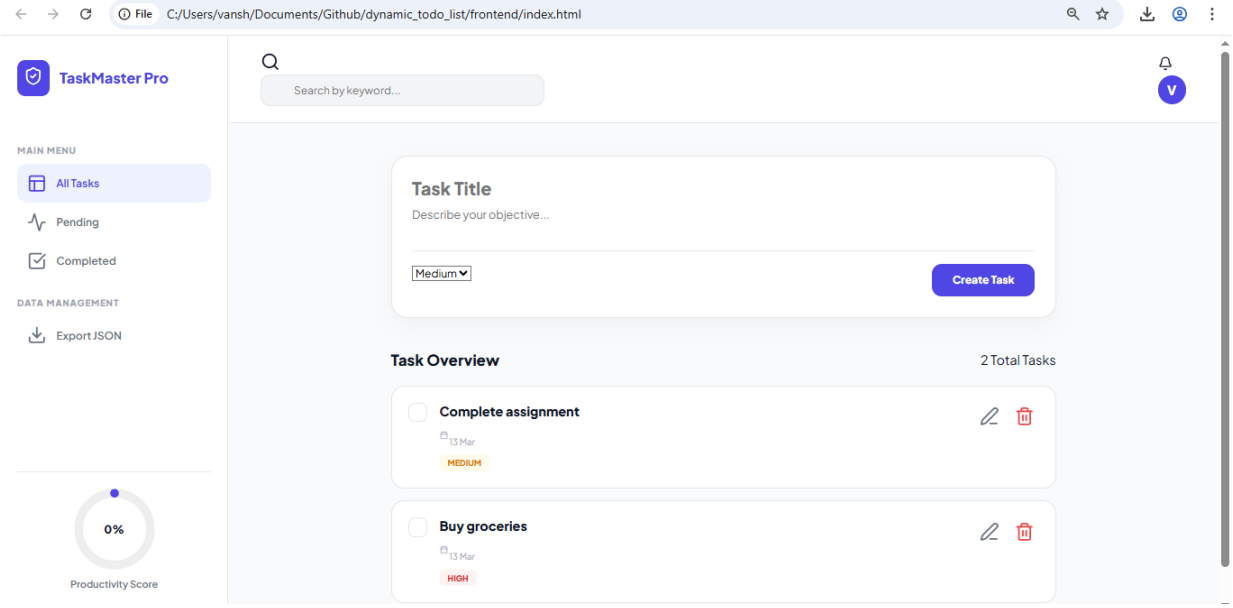
```
});
        res.json(rows);
    });
});

app.post('/tasks', (req, res) => {
    const { title, description, priority } = req.body;
    db.run(`INSERT INTO tasks (title, description, priority)
VALUES (?, ?, ?)`,
    [title, description, priority], function(err) {
        if (err) return res.status(500).json({ error: err.message
});
        res.json({ id: this.lastID, title, description, priority,
isDone: 0 });
    });
});

app.put('/tasks/:id', (req, res) => {
    const { title, priority, description } = req.body;
    db.run(`UPDATE tasks SET title = ?, priority = ?,
description = ? WHERE id = ?`,
    [title, priority, description, req.params.id], function(err) {
        if (err) return res.status(500).json({ error: err.message
});
        res.json({ success: true });
    });
});

app.patch('/tasks/:id/status', (req, res) => {
    db.run(`UPDATE tasks SET isDone = ? WHERE id = ?`,
[req.body.isDone, req.params.id], (err) => {
```

```
        if (err) return res.status(500).json({ error: err.message
});
        res.json({ success: true });
    });
});

app.delete('/tasks/:id', (req, res) => {
    db.run(`DELETE FROM tasks WHERE id = ?`,
req.params.id, (err) => {
        if (err) return res.status(500).json({ error: err.message
});
        res.json({ success: true });
    });
});

app.listen(3000, () => console.log('Server running on
http://localhost:3000'));
```

❖ Output:


TaskMaster Pro
Search by keyword...
MAIN MENU
All Tasks
Pending
Completed
DATA MANAGEMENT
Export JSON
Task Title
Describe your objective...
Medium
Create Task
Task Overview
1 Total Tasks
Complete assignment
13 Mar
MEDIUM
0%
Productivity Score
File
C:/Users/vansh/Documents/Github/dynamic_todo_list/frontend/index.html


TaskMaster Pro
Search by keyword...
MAIN MENU
All Tasks
Pending
Completed
DATA MANAGEMENT
Export JSON
Task Title
Describe your objective...
Medium
Create Task
Task Overview
2 Total Tasks
Complete assignment
13 Mar
MEDIUM
Buy groceries
13 Mar
HIGH
50%
Productivity Score
File
C:/Users/vansh/Documents/Github/dynamic_todo_list/frontend/index.html


TaskMaster Pro
Search by keyword...
MAIN MENU
All Tasks
Pending
Completed
DATA MANAGEMENT
Export JSON
Task Title
Describe your objective...
Medium
Create Task
Task Overview
2 Total Tasks
Complete assignment
13 Mar
MEDIUM
Buy groceries
13 Mar
HIGH
0%
Productivity Score